

FORMATION EMBARQUÉE

Évaluation pratique — Java

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Évaluation pratique — Java (2 h, sur PC)

JDK $\geq$ 17, IDE libre (ou <https://www.onlinegdb.com> pour dépanner). Documents autorisés. Rendu : les `.java` + sortie console des tests. Spécificité Java : on évalue la **modélisation objet** et la **concurrency propre** — pas du C déguisé en Java.

Sujet — Passerelle de mesures

Une passerelle reçoit des trames binaires de capteurs et les expose sous forme d'objets.

Partie A — Modèle objet (/7)

- `record` `Mesure(int idCapteur, double valeur, long horodatageMs)` ;
- interface `Decodeur` : `Optional<Mesure> decoder(byte[] trame);` ;
- deux implémentations : `DecodeurTemperature` (trame `{0xA1, id, hi, lo}` , valeur = mot 16 bits **signé** $\div 10$) et `DecodeurHumidite` (trame `{0xA2, id, v}` , valeur = octet 0..100) ;
- une `Passerelle` qui reçoit un `byte[]` , choisit le bon décodeur d'après l'octet de tête, et renvoie `Optional<Mesure>` .

Points sensibles notés : les masques `& 0xFF` partout (/2), le mot **signé** de la température (contrairement au mini-TP : `(short)` cast à justifier en commentaire) (/2), l'ajout d'un 3^e décodeur **sans modifier** `Passerelle` (expliquer en commentaire comment) (/1), rejets propres via `Optional` — aucune exception pour une trame invalide (/2).

Partie B — Concurrency (/8)

- Un thread « producteur » qui pousse des trames simulées (5/s) dans une `BlockingQueue<byte[]>` **bornée à 32** ;
- un thread « consommateur » qui décode et affiche ;
- arrêt propre des deux threads après 5 s via `interrupt()` + `join()` (le `main` se termine sans `System.exit`) (/3) ;
- la file pleine ne perd pas silencieusement : choisir bloquer OU compter les rejets, et l'écrire en commentaire (/2) ;
- statistiques finales : nombre produit / consommé / rejeté cohérents (/3).

Partie C — Question d'architecture (/5)

En 15 lignes max dans `REPONSES.md` : cette passerelle doit maintenant **historiser en base et exposer un endpoint HTTP**. Découpage en classes/threads proposé, et : pourquoi le callback réseau ne doit-il jamais écrire en base directement ? (*attendu : file + thread dédié — la règle « ISR courte » version serveur, TD 05 ex. 5.*)

Barème

A / 7 · B / 8 · C / 5 — **seuil : 14/20**. Éliminatoire : un `catch (InterruptedException)` vide (le drapeau doit être restauré ou le thread doit sortir).